

BarberTime: Sistema Distribuído de Agendamento em Barbearia com Arquitetura Orientada a Eventos e Aplicativos Móveis Flutter

Caio Victor Kodato Teixeira

Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Curso de Engenharia de Software — Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas (LDAMD)

Belo Horizonte — MG — Brasil

Projeto Integrador — Entrega Final (Sprint 4)

Docentes: Prof. Cleiton Silva Tavares e Prof. Cristiano de Macedo Neto

Tecnologias: Node.js/Express, PostgreSQL, RabbitMQ (AMQP), Flutter/Dart.

Abstract. This document presents the architecture, implementation, and final integration of *BarberTime*, a distributed barbershop-appointment system. The solution comprises a Node.js/Express REST backend with PostgreSQL persistence, a RabbitMQ message-oriented middleware (MOM) used as a topic exchange, and two Flutter mobile applications — one for the client and one for the service provider (barber). The system follows an event-driven architecture: creating a request, accepting it, and rejecting it each publish domain events that decouple the REST command from the notification side effects. The provider application, the core deliverable of this sprint, receives new requests asynchronously through periodic polling with incremental detection of new pending items, and lets the barber accept or reject each request, closing the end-to-end flow client → MOM → provider → client.

Resumo. Este documento apresenta a arquitetura, a implementação e a integração final do *BarberTime*, um sistema distribuído de agendamento em barbearia. A solução é composta por um backend REST em Node.js/Express com persistência em PostgreSQL, um middleware orientado a mensagens (MOM) RabbitMQ usado como *topic exchange* e dois aplicativos móveis Flutter — um para o cliente e outro para o prestador (barbeiro). O sistema adota uma arquitetura orientada a eventos: criar uma solicitação, aceitá-la e recusá-la publicam, cada um, eventos de domínio que desacoplam o comando REST dos efeitos de notificação. O aplicativo do prestador, entrega central desta sprint, recebe novas solicitações de forma assíncrona por *polling* periódico com detecção incremental de novos itens pendentes, e permite ao barbeiro aceitar ou recusar cada pedido, fechando o fluxo de ponta a ponta cliente → MOM → prestador → cliente.

1. Introdução

Sistemas distribuídos modernos raramente se sustentam apenas em chamadas síncronas: à medida que o número de componentes cresce, o acoplamento direto entre serviços torna a evolução custosa e frágil. O Projeto Integrador da disciplina propõe exatamente o exercício desse cenário, exigindo um aplicativo móvel para o cliente, um aplicativo móvel para o prestador, um backend REST e um middleware orientado a mensagens que sustente uma arquitetura orientada a eventos.

O domínio escolhido foi o de **agendamento em barbearia**, que satisfaz naturalmente a distinção entre os dois perfis exigidos. O **cliente** consulta barbeiros e horários livres, solicita um agendamento e acompanha seu status; o **prestador** recebe as solicitações, aceita ou recusa cada pedido e acompanha sua agenda. O ciclo de negócio

central é uma transição de estado da solicitação: criada como pending, ela é levada a confirmed (aceite) ou a cancelled (recusa/cancelamento).

O sistema foi desenvolvido de forma incremental ao longo de quatro sprints: a Sprint 1 entregou a arquitetura e o backend REST; a Sprint 2 integrou o MOM; a Sprint 3 entregou o aplicativo Flutter do cliente; e a Sprint 4, objeto principal deste relatório, entrega o aplicativo do prestador e a integração final de ponta a ponta. Este documento descreve a arquitetura implementada, as decisões de design, as dificuldades encontradas e uma reflexão sobre os padrões estudados.

2. Visão Geral da Arquitetura

O sistema é composto por quatro componentes que se comunicam por dois protocolos: **HTTP/REST**, de natureza síncrona, para comandos e consultas; e **AMQP**, de natureza assíncrona, para a propagação de eventos de domínio por meio do RabbitMQ. A Tabela 1 resume os componentes e suas responsabilidades.

Tabela 1. Componentes do sistema e responsabilidades.

Componente	Tecnologia	Responsabilidade
App Cliente	Flutter/Dart	Solicitar agendamentos e acompanhar o status de forma assíncrona.
App Prestador	Flutter/Dart	Receber solicitações, aceitar/recusar e acompanhar a agenda.
Backend REST	Node.js/Express	Regras de negócio, persistência e publicação de eventos.
MOM	RabbitMQ (topic)	Roteamento assíncrono de eventos entre produtor e consumidores.
Worker de Notificação	Node.js (processo à parte)	Consome eventos e emite notificações (log e e-mail SMTP).
Banco de Dados	PostgreSQL	Persistência de usuários, barbeiros e agendamentos.

A separação entre cliente e prestador, a coexistência de REST e MOM e a divisão em camadas materializam os conceitos de comunicação direta e *comunicação indireta* em sistemas distribuídos discutidos por Coulouris et al. (2011). O worker de notificação executa como processo independente da API, garantindo que a única ponte entre o produtor e o consumidor seja a fila do RabbitMQ.

3. Backend REST e Persistência

3.1. Organização em camadas

O backend adota uma Clean Architecture pragmática, com a regra da dependência apontando para o interior do sistema (Martin, 2019). As rotas (*controllers*) delegam aos casos de uso (*services*), que acessam dados por meio de *repositories*; regras puras, como a grade de horários, ficam isoladas em um pacote de domínio. Os *controllers* não contêm regra de negócio e os *repositories* concentram o SQL parametrizado, de modo que trocar o framework HTTP ou a biblioteca de acesso a dados afetaria apenas as camadas externas.

3.2. Modelo de dados

A persistência usa três tabelas em PostgreSQL: `users` (com `role` em `{client, barber, admin}`), `barbers` (opcionalmente vinculada a um usuário por `user_id`) e `appointments` (com `status` em `{pending, confirmed, cancelled, completed}`). Um índice único parcial sobre (`barber_id`, `starts_at`), restrito aos `status` `pending` e `confirmed`, impede a dupla reserva (*overbooking*) do mesmo horário diretamente no nível do banco.

3.3. Endpoints principais

A API expõe um conjunto de endpoints sob o prefixo `/api/v1`, separados por perfil. As rotas do prestador são protegidas por autenticação JWT e por autorização de papel (`requireRole('barber')`). A Tabela 2 lista os endpoints centrais ao fluxo de negócio.

Tabela 2. Endpoints REST centrais ao fluxo de negócio.

Método	Rota	Perfil	Descrição
POST	<code>/auth/login</code>	público	Autentica e retorna o JWT.
POST	<code>/appointments</code>	cliente	Solicita horário → <code>pending</code> + evento.
GET	<code>/appointments</code>	cliente	Lista os agendamentos do cliente.
PATCH	<code>/appointments/:id/cancel</code>	cliente	Cancela um agendamento.
GET	<code>/barber/appointments</code>	barbeiro	Fila do barbeiro (<code>pending/confirmed/all</code>).
PATCH	<code>/barber/appointments/:id/confirm</code>	barbeiro	Aceita → <code>confirmed</code> + evento.
PATCH	<code>/barber/appointments/:id/reject</code>	barbeiro	Recusa → <code>cancelled</code> + evento.

4. Middleware Orientado a Mensagens e Arquitetura Orientada a Eventos

4.1. Topologia

O MOM é o RabbitMQ, acessado pela biblioteca `amqp1ib` e executado em contêiner Docker. A topologia usa um *topic exchange* durável chamado `barbertime.events`, ao qual estão ligadas filas duráveis, uma por interesse de consumo. O uso de um *topic exchange* com filas distintas concretiza os padrões *Publish-Subscribe Channel* e *Message Router* de Hohpe e Woolf (2003): o produtor desconhece os consumidores e o roteamento é feito pelo *broker* a partir da *routing key*.

4.2. Eventos de domínio

Três eventos de domínio são publicados pelo backend, sempre após a operação REST correspondente ter sido persistida. Cada mensagem é encapsulada em um envelope padronizado (Listing 1) e publicada como JSON persistente. A Tabela 3 descreve os eventos, seus produtores, consumidores e filas.

```
{
  "eventType": "appointment.requested",
  "eventId": "f1e2d3c4-...",
  "timestamp": "2026-06-29T18:30:00.000Z",
  "source": "barbertime-api",
  "payload": { /* dados específicos do evento */ }
}
```

Listing 1. Envelope padronizado de um evento de domínio.

Tabela 3. Eventos de domínio, produtores, consumidores e filas.

Evento	Gatilho (produtor)	Fila / consumidor	Notificado
appointment.requested	POST /appointments	notifications.barber	Barbeiro
appointment.confirmed	PATCH .../confirm	notifications.client	Cliente
appointment.rejected	PATCH .../reject	notifications.client.rejected	Cliente

O Listing 2 mostra o *payload* do evento `appointment.requested`, que carrega os dados necessários para a notificação do barbeiro sem exigir que o consumidor consulte o banco.

```
{
  "appointmentId": "ddddddddd-...",
  "barberId": "11111111-...",
  "barberName": "João Barbeiro",
  "barberEmail": "joao.barbeiro@barbertime.seed",
  "clientId": "aaaaaaaa-...",
  "clientFullName": "Caio Kodato",
  "startsAt": "2026-06-30T12:00:00.000Z",
  "endsAt": "2026-06-30T12:30:00.000Z"
}
```

Listing 2. Payload do evento `appointment.requested`.

4.3. Assincronicidade e desacoplamento

O consumidor (*NotificationConsumer*) executa em um processo separado da API, com `prefetch(1)` e confirmação explícita (`ack/nack`). Não existe chamada REST entre o backend produtor e o worker: a única ligação é a fila. Esse desacoplamento é a essência da *Event-Driven Architecture* descrita por Richardson (2018), na qual serviços reagem a eventos em vez de se invocarem diretamente. Como decisão de projeto, falhas de publicação são registradas em log e não revertem a operação REST, priorizando a disponibilidade do caso de uso principal sobre a entrega garantida da notificação. O Listing 3 ilustra um recorte da saída do worker ao processar uma solicitação.

```
[NOTIFICAÇÃO → BARBEIRO] Nova solicitação de agendamento
Evento:    appointment.requested (f1e2d3c4-...)
Barbeiro:  João Barbeiro <joao.barbeiro@barbertime.seed>
Cliente:   Caio Kodato
Horário:   2026-06-30T12:00:00.000Z → 2026-06-30T12:30:00.000Z
ID agend.: dddddddd-...
```

Listing 3. Recorte do log do worker ao consumir um evento.

5. Aplicativos Móveis Flutter

Ambos os aplicativos seguem a mesma Clean Architecture simplificada, organizada nas camadas *config*, *core*, *models*, *data* e *presentation*, com o fluxo unidirecional *UI* → *AppState* → *Repository* → *ApiClient* → *REST*. A gestão de estado utiliza o pacote *provider*, conforme as boas práticas de estruturação de apps Flutter descritas por Bailey (2023).

5.1. App cliente

O aplicativo cliente, entregue na Sprint 3, oferece telas de listagem, detalhe e criação de agendamentos. A atualização de estado vinda do servidor é assíncrona, por *polling* de dez segundos: quando o barbeiro aceita ou recusa, o status exibido ao cliente muda sem qualquer ação manual.

5.2. App prestador (Sprint 4)

O aplicativo do prestador é a entrega central desta sprint. Ele possui as telas de **Solicitações** (fila de pendentes), **Detalhe da solicitação** (com os botões Aceitar e Recusar), **Em andamento** (confirmados) e **Histórico**. A Tabela 4 relaciona as telas aos requisitos do enunciado da Sprint 4.

Tabela 4. Telas do app prestador × requisitos da Sprint 4.

Requisito	Tela / recurso
Lista de solicitações pendentes	Aba <i>Solicitações</i>
Detalhe com aceitar/recusar	Tela <i>Detalhe da solicitação</i>
Acompanhamento das solicitações em andamento	Abas <i>Em andamento</i> e <i>Histórico</i>
Notificação assíncrona ao prestador	<i>Polling</i> + <i>badge</i> + <i>SnackBar</i>

5.3. Notificação assíncrona ao prestador

A notificação ao prestador não exige ação manual: a fila é recarregada a cada oito segundos. O estado da aplicação guarda o conjunto de identificadores pendentes já conhecidos e, a cada ciclo, compara-o com a lista recém-obtida; identificadores inéditos correspondem a solicitações que chegaram via evento *appointment.requested* e disparam um *badge* numérico e um aviso visual (*SnackBar*). O Listing 4 resume essa lógica em pseudocódigo.

```

a cada 8 segundos:
  lista = GET /barber/appointments?status=all (Bearer JWT)
  pendentes = filtrar(lista, status == "pending" e não passou)
  para cada p em pendentes:
    se p.id não está em conhecidosPendentes:
      enfileirar alerta de nova solicitação
  conhecidosPendentes = ids(pendentes)
  atualizar badge e exibir SnackBar para os novos alertas

```

Listing 4. Detecção incremental de novas solicitações no app prestador.

6. Fluxo Completo de Ponta a Ponta

O caminho principal do sistema integra os quatro componentes de forma assíncrona. O cliente cria uma solicitação por POST /appointments, que persiste o agendamento como pending e publica appointment.requested. O worker consome o evento e notifica o barbeiro. O app prestador, ao recarregar a fila, detecta a nova solicitação e a destaca. Ao aceitar, o backend grava confirmed e publica appointment.confirmed, que leva à notificação do cliente; o app cliente, em seu próprio ciclo de *polling*, passa a exibir o status confirmado. O caminho alternativo da recusa é simétrico, publicando appointment.rejected e liberando o horário. O Listing 5 apresenta os comandos para executar todo o ambiente.

```

# Backend (em backend/)
npm install
npm run db:migrate
docker compose up -d          # RabbitMQ
npm run worker:notifications   # terminal A – consumidor
npm run dev                    # terminal B – API REST

# App cliente
cd mobile_client && flutter pub get && flutter run

# App prestador
cd mobile_provider && flutter pub get && flutter run

```

Listing 5. Comandos de execução do sistema completo.

7. Reflexão sobre os Padrões Estudados

O **REST** define a interface síncrona de comandos e consultas; é simples, *stateless* e cacheável, mas, isolado, exigiria *polling* agressivo ou acoplamento direto para propagar mudanças de estado. O **MOM** e a **arquitetura orientada a eventos** complementam o REST ao desacoplar a notificação do processamento: o produtor publica um fato e quem se interessa reage, o que melhora a evolutividade — novos consumidores, como envio de *push* ou coleta de métricas, podem ser acrescentados sem alterar o produtor (Hohpe e Woolf, 2003; Richardson, 2018).

A **Clean Architecture** mantém as regras de negócio independentes de framework, banco e interface, tanto no backend quanto nos aplicativos (Martin, 2019); foi ela que permitiu reaproveitar a estrutura do app cliente no app prestador com baixo esforço. Por fim, o projeto materializa conceitos de **sistemas distribuídos**: comunicação direta (REST) e indireta (filas), além de tolerância parcial a falhas, já que a indisponibilidade do MOM não derruba a API (Coulouris et al., 2011).

8. Dificuldades Encontradas e Soluções Adotadas

A primeira dificuldade foi que o backend originalmente só permitia o aceite. Como o enunciado exige aceitar *e* recusar, foi necessário criar o endpoint `PATCH /barber/appointments/:id/reject`, o evento `appointment.rejected`, a fila e o *binding* correspondentes e o respectivo tratamento no worker, mantendo o mesmo padrão já usado no aceite.

A segunda dificuldade foi notificar o prestador sem infraestrutura de *push*. Em vez de integrar um serviço externo, optou-se por *polling* com detecção incremental de novos identificadores, suficiente para a notificação assíncrona exigida e operável sem dependências adicionais. Outras decisões incluíram padronizar o fuso horário (persistência em UTC e exibição em *America/Sao_Paulo*) para evitar divergências entre emuladores, e criar o app prestador com identificador e nome próprios para coexistir com o app cliente sem conflito, reutilizando a identidade visual com um selo distintivo.

9. Decisões e Justificativas Tecnológicas

A Tabela 5 sintetiza as principais escolhas tecnológicas e suas justificativas.

Tabela 5. Decisões tecnológicas e justificativas.

Aspecto	Escolha	Justificativa
Backend	Node.js + Express	Leve, ecossistema maduro, alinhado à ementa.
Banco	PostgreSQL	Integridade relacional e índice único parcial contra overbooking.
MOM	RabbitMQ (topic)	Roteamento por routing key, filas duráveis e painel de gestão.
Apps	Flutter/Dart	Código único multiplataforma; estado com provider.
Atualização nos apps	Polling	Simplicidade e robustez em rede móvel, sem conexão persistente.
Contêiner	Docker (RabbitMQ)	Sobe o broker isolado, sem instalar AMQP no host.

10. Conclusão

O BarberTime cumpre o escopo obrigatório do Projeto Integrador: dois aplicativos móveis (cliente e prestador), um backend REST com persistência e um middleware orientado a mensagens que sustenta uma arquitetura orientada a eventos real. O fluxo de ponta a ponta — solicitação, notificação assíncrona, aceite ou recusa e atualização do cliente — funciona de forma desacoplada e demonstrável. A adoção consistente de Clean Architecture no backend e nos dois aplicativos tornou o sistema legível, testável e fácil de estender, validando na prática os padrões estudados ao longo da disciplina. Como trabalhos futuros, destacam-se a substituição do *polling* por notificações *push* e a adição de novos consumidores de eventos (métricas e histórico de auditoria) sem alteração do produtor, justamente o tipo de evolução que a arquitetura orientada a eventos favorece.

Referências

COULOURIS, G. et al. *Distributed Systems: concepts and design*. 5th ed. Boston: Addison-Wesley, 2011.

HOHPE, G.; WOOLF, B. *Enterprise Integration Patterns: designing, building, and deploying messaging solutions*. Boston: Addison-Wesley, 2003.

MARTIN, R. C. *Arquitetura limpa: o guia do artesão para estrutura e design de software*. Rio de Janeiro: Alta Books, 2019.

RICHARDSON, C. *Microservices patterns: with examples in Java*. Shelter Island: Manning, 2018.

BAILEY, T. *Flutter for beginners*. 3rd ed. Birmingham: Packt, 2023.